

Reporte de Implementación del Patrón Interpreter en el Sistema de Citas Médicas del ISSSTE

Introducción

Este documento presenta la implementación del patrón Interpreter en el sistema de gestión de citas médicas del ISSSTE. El patrón Interpreter fue seleccionado para mejorar la validación de reglas de negocio complejas relacionadas con formatos de documentos, horarios médicos y especialidades. La implementación busca centralizar y hacer más mantenible la lógica de validación que actualmente está dispersa en el código.

¿Qué es el Patrón Interpreter?

El patrón Interpreter es un patrón de diseño conductual que proporciona una manera de evaluar expresiones o gramáticas de un lenguaje. Es particularmente útil para:

- Definir una gramática para un lenguaje
- Representar reglas de negocio complejas como árboles de objetos
- Implementar sistemas que necesiten interpretar expresiones

En el contexto de nuestro sistema, el patrón nos permite tratar cada regla de validación como un objeto independiente que puede ser combinado con otros para formar validaciones más complejas.

Participantes del Patrón Interpreter

En nuestra implementación, los participantes del patrón son:

1. **AbstractExpression (Expression):** Interfaz que declara el método `interpret()`
2. **TerminalExpressions:**
 - **DateFormatExpression:** Valida formatos de fecha
 - **TimeFormatExpression:** Valida horarios médicos
 - **SpecialtyExpression:** Valida especialidades médicas
 - **DocumentExpression:** Valida formatos de CURP y RFC
3. **NonTerminalExpression (AndExpression):** Combina expresiones para validaciones complejas
4. **Context:** Las cadenas de texto a validar (fechas, horarios, documentos)
5. **Client:** Las clases del sistema que usan las expresiones (RegistrarUsuario, Formato_Citas)

Implementación en el Proyecto

1. Estructura Básica del Interpreter

```
// Interfaz Expression
package Patrones.Interpreter;

public interface Expression {
    boolean interpret(String context);
}
```

```
// Implementación de DocumentExpression
package Patrones.Interpreter;
import java.util.regex.Pattern;

public class DocumentExpression implements Expression {
    private final DocumentType type;

    public DocumentExpression(DocumentType type) {
        this.type = type;
    }

    @Override
    public boolean interpret(String context) {
        switch(type) {
            case CURP:
                return Pattern.matches("[A-Z]{4}[0-9]{6}[A-Z]{6}[0-9A-Z]{2}", context);
            case RFC:
                return Pattern.matches("[A-Z]{4}[0-9]{6}[A-Z0-9]{3}", context);
            default:
                return false;
        }
    }

    public enum DocumentType { CURP, RFC }
}
```

2. Integración con el Sistema Existente

```
// En RegistrarUsuario.java
public static boolean registrar(Connection conn, Usuario usuario) throws
SQLException {
    // Validación con Interpreter
    DocumentExpression curpValidator = new
DocumentExpression(DocumentExpression.DocumentType.CURP);
    if(!curpValidator.interpret(usuario.getCurp())) {
        JOptionPane.showMessageDialog(null, "Formato de CURP inválido",
"Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
    // Resto del método...
}

// En Formato_Citas.java
```

```

public static void obtenerHorasDisponibles(Connection conn,
JComboBox<String> horaComboBox,
                                String fecha, int idMedico) {
    TimeFormatExpression timeValidator = new
TimeFormatExpression("HH:mm");
    // Resto del método...
}

```

3. Expresiones Compuestas

```

// Validación compuesta en algún servicio
Expression validUserRegistration = new AndExpression(
    new DocumentExpression(DocumentExpression.DocumentType.CURP),
    new AndExpression(
        new DocumentExpression(DocumentExpression.DocumentType.RFC),
        new DateFormatExpression("yyyy-MM-dd")
    )
);

if(validUserRegistration.interpret(userData)) {
    // Registrar usuario
}

```

Diagrama UML

Beneficios en el Proyecto

1. Validación Centralizada: Todas las reglas de validación están encapsuladas en clases específicas
2. Flexibilidad: Nuevas reglas pueden añadirse sin modificar código existente
3. Legibilidad: Las reglas de negocio son más fáciles de entender al estar representadas como objetos
4. Mantenibilidad: Cambios en formatos o reglas afectan solo a una clase
5. Combinabilidad: Las reglas pueden combinarse para formar validaciones complejas
6. Consistencia: Mismos estándares de validación en toda la aplicación

Conclusión

La implementación del patrón Interpreter en el sistema de citas médicas del ISSSTE ha demostrado ser una solución elegante para el problema de validación de reglas de negocio. El patrón permitió:

- Reducir código duplicado de validación
- Mejorar la capacidad de mantenimiento
- Facilitar la adición de nuevas reglas
- Hacer el código más expresivo y legible

La arquitectura resultante es más robusta y preparada para futuras expansiones, especialmente en áreas como:

- Validación de nuevos documentos médicos
- Soporte para diferentes formatos de fecha/hora
- Expansión de especialidades médicas

Esta implementación sirve como base para futuras mejoras en el sistema, pudiendo extenderse a otras áreas como interpretación de diagnósticos médicos o procesamiento de informes clínicos.